

T2: Matching de Produtos (Relatório Final)

Disciplina: Aprendizado de Máquina

Professor: Otavio Parraga

Integrantes: Rafael Magalhaes (25180166) e Pedro Martini Lehn (22280113)

Data: Junho de 2026

Resumo. Este trabalho constrói e compara pipelines automáticos de matching de produtos para o problema real da Nubo Desenvolvimento de Software: dado um pedido de compra escrito livremente (ex.: "COCA COLA 1L C/6"), identificar o produto correspondente em um catálogo normalizado de 14.206 itens. Implementamos a abordagem clássica de recuperação de informação (TF-IDF e BM25) e duas estratégias de deep learning (embeddings semânticos e LLM como reranker), avaliadas por P@1, MRR@5 e R@5. No conjunto de teste, a abordagem clássica atingiu P@1 = 99,2% e a abordagem com LLM, P@1 = 99,6%, com os erros remanescentes explicados por ambiguidades do próprio catálogo.

1. Referencial Teórico

1.1 O problema: recuperação de informação

O matching de produtos é um caso clássico de **recuperação de informação** (*information retrieval*): dado um texto de consulta (a *query*), ranquear documentos de uma coleção (os produtos do catálogo) por relevância (MANNING; RAGHAVAN; SCHUTZE, 2008). A saída não é uma única resposta, mas uma **lista ranqueada**, o que motiva as métricas adotadas.

1.2 TF-IDF

O **TF-IDF** (*Term Frequency, Inverse Document Frequency*) atribui a cada termo de um documento um peso que combina duas intuições: termos frequentes no documento o descrevem bem (TF), e termos raros na coleção como um todo são mais informativos (IDF). Em termos econômicos, é o princípio da escassez aplicado a palavras: "garrafa" aparece em milhares de produtos e vale pouco para distinguir um deles; "gastronomique" aparece em pouquíssimos e vale muito. Cada texto vira um vetor esparso com um peso por termo do vocabulário.

Neste trabalho usamos também TF-IDF sobre **n-gramas de caracteres** (sequências de 3 a 5 letras), técnica que torna a comparação robusta a erros de grafia e flexões ("açúcar" e "açúcares" compartilham quase todos os fragmentos).

1.3 Similaridade de cosseno

Com os textos representados como vetores, a **similaridade de cosseno** mede o cosseno do ângulo entre eles: 1 quando apontam na mesma direção (mesma "receita" de termos), 0 quando não compartilham termo algum. Por ignorar o comprimento dos vetores, compara a *composição proporcional* dos textos, analogamente a comparar a composição percentual de duas cestas de consumo, e não seus valores absolutos.

1.4 BM25

O **BM25** é uma função de ranqueamento probabilística, padrão em motores de busca como Lucene e Elasticsearch. Refina o TF-IDF com duas ideias: **saturação de frequência** (a segunda ocorrência de um termo no documento contribui menos que a primeira, exatamente como utilidade marginal decrescente) e **normalização pelo tamanho do documento**, evitando que textos longos ganhem vantagem indevida.

1.5 Embeddings semânticos

Um **embedding** é uma representação densa de um texto: um vetor de algumas centenas de números reais que captura seu significado, aprendido por uma rede neural treinada em grandes volumes de texto. Textos com sentidos próximos ficam próximos no espaço vetorial mesmo sem palavras em comum. Para o estudante de economia, a analogia natural é a teoria do consumidor de Lancaster: um produto não é seu nome, mas um

pacote de características; o embedding posiciona cada produto num espaço de características aprendidas automaticamente. Usamos modelos da família **Sentence-Transformers**, treinados especificamente para que a similaridade de cosseno entre embeddings de frases reflita similaridade semântica, incluindo o **E5 multilíngue**, otimizado para tarefas de busca.

1.6 LLMs e aprendizado em contexto

Grandes modelos de linguagem (LLMs), baseados na arquitetura Transformer, são capazes de **aprendizado em contexto**: executam tarefas novas a partir de instruções em linguagem natural, sem nenhum exemplo (**zero-shot**) ou com poucos exemplos resolvidos incluídos no prompt (**few-shot**). Neste trabalho, o LLM atua como **reranker**: recebe a query e uma lista curta de candidatos pré-filtrados e decide o ranking final, combinando a eficiência da busca clássica com a capacidade de raciocínio do modelo.

1.7 Métricas de avaliação

P@1 (Precision@1): fração de queries em que o produto correto ficou em 1º lugar.

MRR@5 (Mean Reciprocal Rank): média de $1/\text{posição}$ do produto correto no top-5 (0 se ausente); dá crédito parcial a acertos nas posições 2 a 5.

R@5 (Recall@5): fração de queries em que o produto correto aparece em alguma das 5 primeiras posições.

2. Delimitação do Desenvolvimento

Reaproveitado (bibliotecas públicas, uso documentado):

`scikit-learn`, vetorização TF-IDF (`TfidfVectorizer`) e produto interno (`linear_kernel`);

`Rank_bm25`, implementação do BM25Okapi;

`sentence-transformers` + modelos públicos do Hugging Face (`paraphrase-multilingual-MiniLM-L12-v2`, `intfloat/multilingual-e5-small`);

SDKs oficiais das APIs (`google-genai`, `anthropic`);

`pandas`, `numpy`, `matplotlib` (manipulação de dados e gráficos).

Nenhum trecho de código foi copiado de repositórios de terceiros além do uso convencional dessas bibliotecas.

Implementado pelo grupo:

Módulo de pré-processamento textual com as regras derivadas da exploração dos dados (`comum/preprocessamento.py`);

Script de avaliação com as três métricas exigidas (`comum/avaliacao.py`);

Pipelines completos das duas abordagens, incluindo a busca em grade de variações na validação (Etapas 2 e 3);

Pipeline híbrido LLM-reranker (filtro top-10 + prompts zero/few-shot + saída estruturada em JSON Schema), com sistema de checkpoint retomável e tratamento de limites de API;

Análises comparativas (por tipo de query, sobreposição de erros, oráculo) e o experimento NO_MATCH com grupo de controle;

Todos os experimentos, gráficos e este relatório.

3. Metodologia

3.1 Dados e protocolo de avaliação

Arquivo	Conteúdo	Uso
catalog.csv	14.206 produtos (id EAN/GTIN, nome, marca)	espaço de busca
queries.csv	16.441 queries sem gabarito	exploração e desenvolvimento
queries_val.csv	250 queries anotadas	seleção de TODAS as configurações

Arquivo	Conteúdo	Uso
queries_test.csv	250 queries anotadas	avaliação final, uso único

O conjunto de teste permaneceu intocado durante todo o desenvolvimento: cada abordagem teve sua configuração **congelada com base apenas na validação** e avaliada uma única vez no teste. Detalhe técnico relevante: os códigos EAN começam com zero e foram lidos como texto para preservá-lo.

A exploração inicial (Etapa 1) revelou dois perfis de query, a maioria "limpa" (ex.: "Feijão Preto Tipo 1 Qualitá Pacote 1kg") e uma minoria "de distribuidor" (ex.: "PITU LATAO 473ml C/12"), além de 966 produtos-base com múltiplas variações de volume no catálogo e produtos duplicados com nomes idênticos.

3.2 Pré-processamento

Aplicado igualmente a queries e nomes de catálogo: minúsculas; remoção de acentos e pontuação; remoção de termos de embalagem do pedido (C/6, C/12, UND, CX, FD, PCT), que descrevem o pedido e não o produto; padronização de unidades coladas ao número (1 litro -> 1l; 15 GR -> 15g), preservando volumes e pesos por serem discriminativos entre variações; conversão de "S/" em "sem"; remoção de stopwords leves. **Divergência justificada do enunciado:** mantivemos "com" e "sem" no vocabulário; removê-las tornaria "água com gás" e "água sem gás" idênticas.

3.3 Abordagem 1 (NLP clássico)

Para cada query, o pipeline calcula a similaridade com os 14.206 produtos e retorna o top-5 com pontuações. Testamos na validação 8 variações: {TF-IDF palavra (1,1); TF-IDF palavra (1,2); TF-IDF caracteres (3,5); BM25} x {indexar só o nome; nome + marca}. Configuração campeã (congelada): **TF-IDF de caracteres (3,5), indexando apenas o nome.**

3.4 Abordagem 2 (Deep Learning)

Estratégia A (Embeddings): geramos embeddings do catálogo e das queries com dois modelos multilíngues (MiniLM e E5-small), em texto cru e pré-processado (4 variações), ranqueando por cosseno. Campeã: E5-small com texto pré-processado.

Estratégia B, LLM como reranker (modelo final da abordagem): pipeline híbrido recomendado no enunciado, o TF-IDF campeão filtra os 10 candidatos mais prováveis (com R@5 = 100% na validação, o correto quase sempre está entre eles) e o LLM decide o ranking final, em modo zero-shot e few-shot (3 exemplos do próprio enunciado, sem contaminar a validação).

Iniciamos com a API gratuita do Google Gemini, como sugerido. Em junho de 2026, porém, o nível gratuito limita cada modelo a **20 requisições/dia**, o que forçou processamento em lotes de 32 queries por chamada, e o modelo estável apresentou indisponibilidade crônica (erros 503) por dois dias seguidos. Concluímos a validação no gemini-2.5-flash-lite e, em seguida, **migramos a estratégia para a API paga da Anthropic** (claude-haiku-4-5, custo total de aproximadamente US\$ 0,85 em créditos), que permitiu o desenho mais limpo: uma chamada por query e **saída estruturada com JSON Schema** (formato de resposta garantido pela API). Por consistência, validação e teste da configuração final usaram o mesmo modelo. Configuração congelada: **Claude Haiku 4.5, few-shot, reranking dos top-10 do TF-IDF.**

3.5 Medições

Além das métricas de qualidade, medimos tempo de execução para as 250 queries e custo monetário de cada sistema, dimensões exigidas na tabela comparativa.

4. Resultados e Análise

4.1 Validação (250 queries), seleção de configurações

Sistema	P@1	MRR @5	R@5	Tempo (250 q)	Custo
Claude few-shot (reranker)	99,2%	0,995	100 %	13,6 min	~US\$ 0,30

Sistema	P@1	MRR@5	R@5	Tempo (250 q)	Custo
Claude zero-shot (reranker)	99,2%	0,995	100%	18,5 min	~US\$ 0,25
TF-IDF caracteres (3,5)	98,4%	0,990	100%	1,3 s	gratuito
Gemini few-shot (reranker)	98,0%	0,990	100%	1,9 min	gratuito (20 req/dia)
BM25 (nome+marca)	97,6%	0,988	100%	4,6 s	gratuito
Gemini zero-shot (reranker)	96,8%	0,983	100%	1,8 min	gratuito (20 req/dia)
Embeddings E5-small	95,2%	0,974	100%	57 s	gratuito (CPU)

Observações: (i) todos os sistemas retornam lista top-5 ranqueada, então P@1, MRR@5 e R@5 foram calculados integralmente; (ii) no Gemini, os 3 exemplos few-shot valeram +1,2 p.p.; no Claude, zero-shot e few-shot empataram (modelos mais capazes extraem as regras da própria instrução); (iii) o pré-processamento beneficiou até a rede neural: o MiniLM saltou de 51,6% (texto cru) para 91,2% (pré-processado).

4.2 Teste (250 queries), métricas finais

Abordagem	P@1	MRR@5	R@5	Tempo (250 q)	Custo	Complexidade
2, Deep Learning (Claude few-shot)	99,6%	0,998	100%	12,7 min	~US\$ 0,30/250 q	Média-Alta
1, TF-IDF / BM25 (TF-IDF char)	99,2%	0,996	100%	1,4 s	gratuito	Baixa
(referência) BM25	98,8%	0,992	100%	4,7 s	gratuito	Baixa
(referência) Embeddings E5	98,4%	0,989	99,6%	65,5 s	gratuito	Média

Todos os sistemas foram iguais ou melhores no teste do que na validação, nenhum sinal de sobreajuste do protocolo de seleção. Com $n = 250$, diferenças de ~ 1 p.p. estão dentro do ruído estatístico.

4.3 Em quais queries cada abordagem se sai melhor

Por segmento (validação): embeddings lideravam sozinhos nas queries longas (100% em ≥ 10 palavras), mas sofrem nas curtas e nas em maiúsculas (85,7%); o clássico domina onde a decisão é lexical (curtas, volume/peso); o LLM não tem ponto fraco em nenhum segmento. Análise de sobreposição: 18 queries foram erradas por pelo menos um sistema, **nenhuma por todos**; um "oráculo" que escolhesse o sistema certo por query acertaria 100% da validação, o que motiva a proposta de ensemble (seção 5).

4.4 Análise qualitativa

Acertos. O caso de uso central é resolvido com folga pelos dois finalistas: "VODKA ORLOFF 1L" -> "Vodka orloff 1 litro"; "TANQUERAY GIN 750ML UNID" -> "Gin tanqueray ten 750ml" (abreviações descartadas, unidades casadas, ordem de palavras irrelevante).

Erros (duas categorias). (1) *Variação fina que exige raciocínio*, onde o LLM corrige o clássico: o TF-IDF respondeu o sabonete de 900ml para a query "... Sachê 200ml Refil" e a manteiga "extra com sal" para a query "Extra sem Sal"; o Claude acertou ambos priorizando volume e o par com/sem. (2) *Ambiguidade dos dados*, que nenhum sistema resolve: a query "Pipoca ... Premium 90g" não tem variante exata no catálogo (há premium 50g, light 90g e natural 100g); a query do creme dental mistura os nomes de duas variantes reais do produto; o único erro do Claude no teste foi um vinho cuja query diz apenas "Tinto" para uma vinícola com cinco tintos no catálogo.

Casos ambíguos do catálogo. Documentamos duplicatas verdadeiras: dois cadastros de "Adoçante Líquido sucralose linea 75ml" com nomes idênticos e IDs distintos; manteigas Président com as mesmas palavras em

ordem trocada; oito vinhos "Cordero con Piel de Lobo". Essas duplicatas estabelecem o teto de ~99%; os erros remanescentes são dos dados, não dos modelos.

Casos NO_MATCH. Selecionamos de queries.csv 25 queries cujo termo de marca não existe no catálogo (ex.: "DEVASSA LITRINHO C/24"). A abordagem clássica nunca se recusa a responder (retorna o mais parecido, mesmo absurdo), mas seu score a denuncia: mediana 0,54 nos NO_MATCH contra 0,96 nas queries com correspondência; um limiar no percentil 5 da validação detecta 23/25 com 5% de falso alarme. O LLM, instruído a poder abster-se, absteve-se em 52% dos casos com **zero abstenção indevida** num grupo de controle de 25 queries com correspondência (e 24/25 de acerto nele). Inspecionando os casos sem abstenção, constatamos que a maioria era falha da nossa heurística de rotulagem; eram produtos que existiam no catálogo e o modelo os encontrou corretamente (ex.: o shampoo Pampers). Conclusão dupla: o desempenho real de detecção é maior que os números brutos, e rotular NO_MATCH automaticamente é, por si só, um problema difícil.

4.5 Tradeoff simplicidade x desempenho

A resposta tem dois capítulos. **Contra o LLM gratuito, a simplicidade vence:** o Gemini gratuito (98,0%) perdeu para o TF-IDF (98,4%) sendo ~90x mais lento, limitado a 20 requisições/dia e sujeito a indisponibilidade crônica. **Contra o LLM pago, o desempenho tem preço conhecido:** o Claude comprou +0,8 p.p. na validação e +0,4 p.p. no teste por ~US\$ 0,30 a cada 250 queries; no volume real da Nubo (milhares de pedidos/dia), cerca de US\$ 19/dia, contra custo zero do clássico, que é ~600x mais rápido e mantém os dados na máquina. A tarefa é majoritariamente de casamento lexical fino, terreno do clássico; o LLM agrega valor na cauda de casos que exigem raciocínio (volumes implícitos, pares com/sem, abstenção em NO_MATCH).

5. Conclusão, Melhorias e Dificuldades

5.1 Principais aprendizados

- 1 Para matching de textos curtos com vocabulário técnico, **métodos clássicos bem ajustados são extremamente competitivos** (99,2% no teste, custo zero); n-gramas de caracteres foram decisivos contra erros de grafia.
- 2 **Pré-processamento orientado pelos dados beneficia todas as abordagens**, inclusive as neurais (MiniLM: 51,6% -> 91,2%).
- 3 O **LLM-reranker híbrido** une o melhor dos dois mundos: a velocidade do filtro clássico e o raciocínio fino do modelo, e ainda sabe **abster-se** em casos sem resposta, capacidade inexistente no clássico puro.
- 4 A disciplina validação x teste funcionou: hierarquia preservada e nenhum sobreajuste.
- 5 O teto de desempenho é dos **dados**: deduplicar o catálogo renderia mais que trocar de modelo.

5.2 Dificuldades

A principal foi a **infraestrutura gratuita de LLMs em 2026**: cotas de 20 requisições/dia por modelo e erros 503 crônicos no nível gratuito do Gemini, que exigiram processamento em lotes, sistema de checkpoints retomáveis e, por fim, migração para API paga, uma amostra realista das restrições operacionais que o problema enfrentaria em produção com orçamento zero. Dificuldades menores incluíram a preservação de zeros à esquerda nos códigos EAN e a codificação de caracteres no ambiente Windows.

5.3 Melhorias futuras

- **Ensemble por margem de confiança:** usar o LLM apenas quando a diferença de score entre o 1º e o 2º candidato do TF-IDF for pequena, qualidade de LLM a ~10% do custo (a análise do oráculo indica teto de 100% na validação);
- **Deduplicação do catálogo** e enriquecimento dos cadastros (a maior fonte de erro remanescente);
- **Fine-tuning contrastivo** de um modelo de embeddings com pares (query, produto), os 16 mil pares de queries.csv, uma vez anotados, seriam o conjunto de treino natural;
- **Detector de NO_MATCH em produção**, combinando limiar de score e abstenção do LLM;
- **Cache de queries repetidas** (1.688 duplicatas em queries.csv) para reduzir custo e latência;

• Dashboard gerencial (ex.: Power BI) para acompanhamento de acurácia e custos em produção.

6. Referências

MANNING, C. D.; RAGHAVAN, P.; SCHUTZE, H. Introduction to Information Retrieval. Cambridge: Cambridge University Press, 2008.

Documentação técnica: scikit-learn (TfidfVectorizer), rank-bm25, Sentence-Transformers, Google Gemini API, Anthropic Claude API.